

```

<!--
{
  "availability" : [
    "iOS: 8.0.0 -",
    "iPadOS: 8.0.0 -",
    "macCatalyst: 13.0.0 -",
    "macOS: 10.10.0 -",
    "tvOS: 9.0.0 -",
    "visionOS: 1.0.0 -",
    "watchOS: 2.0.0 -"
  ],
  "documentType" : "symbol",
  "framework" : "Swift",
  "identifier" : "/documentation/Swift/Hashable",
  "metadataVersion" : "0.1.0",
  "role" : "Protocol",
  "symbol" : {
    "kind" : "Protocol",
    "modules" : [
      "Swift"
    ]
  },
  "preciseIdentifier" : "s:SH"
},
{
  "title" : "Hashable"
}
-->

```

Hashable

A type that can be hashed into a `Hasher` to produce an integer hash value.

```

...

```

```

protocol Hashable : Equatable, ~Copyable, ~Escapable
{
}

```

Overview

You can use any type that conforms to the `Hashable` protocol in a set or as a dictionary key. Many types in the standard library conform to `Hashable`: Strings, integers, floating-point and Boolean values, and even sets are hashable by default. Some other types, such as optionals, arrays and ranges automatically become hashable when their type arguments implement the same.

Your own custom types can be hashable as well. When you define an enumeration without associated values, it gains `Hashable` conformance automatically, and you can add `Hashable` conformance to your other custom types by implementing the `hash(into:)` method. For structs whose stored properties are all `Hashable`, and for enum types that have all `Hashable` associated values, the compiler is able to provide an implementation of `hash(into:)` automatically.

Hashing a value means feeding its essential components into a hash function, represented by the `Hasher` type. Essential components are those that contribute to the type's implementation of `Equatable`. Two instances that are equal must feed the same values to `Hasher` in `hash(into:)`, in the same order.

Conforming to the Hashable Protocol

To use your own custom type in a set or as the key type of a dictionary, add `Hashable` conformance to your type. The `Hashable` protocol inherits from the `Equatable` protocol, so you must also satisfy that protocol's

requirements.

The compiler automatically synthesizes your custom type's ``Hashable`` and requirements when you declare ``Hashable`` conformance in the type's original declaration and your type meets these criteria:

- For a ``struct``, all its stored properties must conform to ``Hashable``.
- For an ``enum``, all its associated values must conform to ``Hashable``. (An ``enum`` without associated values has ``Hashable`` conformance even without the declaration.)

To customize your type's ``Hashable`` conformance, to adopt ``Hashable`` in a type that doesn't meet the criteria listed above, or to extend an existing type to conform to ``Hashable``, implement the ``hash(into:)`` method in your custom type.

In your ``hash(into:)`` implementation, call ``combine(_:)`` on the provided ``Hasher`` instance with the essential components of your type. To ensure that your type meets the semantic requirements of the ``Hashable`` and ``Equatable`` protocols, it's a good idea to also customize your type's ``Equatable`` conformance to match.

As an example, consider a ``GridPoint`` type that describes a location in a grid of buttons. Here's the initial declaration of the ``GridPoint`` type:

```
...  
/// A point in an x-y coordinate system.  
struct GridPoint {  
    var x: Int  
    var y: Int  
}  
...
```

You'd like to create a set of the grid points where a user has already tapped. Because the ``GridPoint`` type is not hashable yet, it can't be used in a set. To add ``Hashable`` conformance, provide an ``==`` operator function and implement the ``hash(into:)`` method.

```
...  
extension GridPoint: Hashable {  
    static func == (lhs: GridPoint, rhs: GridPoint) -> Bool {  
        return lhs.x == rhs.x && lhs.y == rhs.y  
    }  
  
    func hash(into hasher: inout Hasher) {  
        hasher.combine(x)  
        hasher.combine(y)  
    }  
}  
...
```

The ``hash(into:)`` method in this example feeds the grid point's ``x`` and ``y`` properties into the provided hasher. These properties are the same ones used to test for equality in the ``==`` operator function.

Now that ``GridPoint`` conforms to the ``Hashable`` protocol, you can create a set of previously tapped grid points.

```
...  
var tappedPoints: Set = [GridPoint(x: 2, y: 3), GridPoint(x: 4, y: 1)]  
let nextTap = GridPoint(x: 0, y: 1)  
if tappedPoints.contains(nextTap) {
```

```

        print("Already tapped at \(nextTap.x), \(nextTap.y)).")
    } else {
        tappedPoints.insert(nextTap)
        print("New tap detected at \(nextTap.x), \(nextTap.y)).")
    }
}
// Prints "New tap detected at (0, 1).")

```

Topics

Providing a Hash Value

`[`hash(into:)]`(/documentation/Swift/Hashable/hash(into:))

Hashes the essential components of this value by feeding them into the given hasher.

Deprecated

`[`hashValue`]`(/documentation/Swift/Hashable/hashValue)

The hash value.

Relationships

Conforming Types

`[`AnyKeyPath`]`(/documentation/Swift/AnyKeyPath)

`[`SIMD16`]`(/documentation/Swift/SIMD16)

`[`UnsafeCurrentTask`]`(/documentation/Swift/UnsafeCurrentTask)

`[`UInt32`]`(/documentation/Swift/UInt32)

`[`Index`]`(/documentation/Swift/LazyPrefixWhileSequence/Index)

`[`Index`]`(/documentation/Swift/FlattenSequence/Index)

`[`ContiguousArray`]`(/documentation/Swift/ContiguousArray)

`[`CanonicalCombiningClass`]`(/documentation/Swift/Unicode/CanonicalCombiningClass)

`[`CollectionDifference`]`(/documentation/Swift/CollectionDifference)

`[`Int128`]`(/documentation/Swift/Int128)

`[`FloatingPointClassification`]`(/documentation/Swift/FloatingPointClassification)

`[`Index`]`(/documentation/Swift/ClosedRange/Index)

`[`UnitWidth-swift.struct`]`(/documentation/Swift/Duration/UnitsFormatStyle/UnitWidth-swift.struct)

`[`StandardComparator`]`(/documentation/Swift/String/StandardComparator)

`[`Substring`]`(/documentation/Swift/Substring)

`[`Attributed-swift.struct`]`(/documentation/Swift/Duration/UnitsFormatStyle/Attributed-swift.struct)

[`Never`](/documentation/Swift/Never)

[`SIMD32`](/documentation/Swift/SIMD32)

[`PartialKeyPath`](/documentation/Swift/PartialKeyPath)

[`Instant`](/documentation/Swift/SuspendingClock/Instant)

[`Set`](/documentation/Swift/Set)

[`ReferenceWritableKeyPath`](/documentation/Swift/ReferenceWritableKeyPath)

[`Index`](/documentation/Swift/String/Index)

[`CodingUserInfoKey`](/documentation/Swift/CodingUserInfoKey)

[`RegexSemanticLevel`](/documentation/Swift/RegexSemanticLevel)

[`ObservationRegistrar`](/documentation/Observation/ObservationRegistrar)

[`Array`](/documentation/Swift/Array)

[`RegexWordBoundaryKind`](/documentation/Swift/RegexWordBoundaryKind)

[`Pattern-swift.struct`](/documentation/Swift/Duration/TimeFormatStyle/Pattern-swift.struct)

[`WordPair`](/documentation/Synchronization/WordPair)

[`DiscontiguousSlice`](/documentation/Swift/DiscontiguousSlice)

[`AutoreleasingUnsafeMutablePointer`](/documentation/Swift/AutoreleasingUnsafeMutablePointer)

[`TimeFormatStyle`](/documentation/Swift/Duration/TimeFormatStyle)

[`RegexRepetitionBehavior`](/documentation/Swift/RegexRepetitionBehavior)

[`NumericType`](/documentation/Swift/Unicode/NumericType)

[`Range`](/documentation/Swift/Range)

[`Continuation`](/documentation/Swift/AsyncThrowingStream/Continuation)

[`ErrorCode-swift.enum`](/documentation/Distributed/ExecuteDistributedTargetError/ErrorCode-swift.enum)

[`KeyboardType-swift.enum`](/documentation/Swift/String/IntentInputOptions/KeyboardType-swift.enum)

[`UInt64`](/documentation/Swift/UInt64)

[`WritableKeyPath`](/documentation/Swift/WritableKeyPath)

[`RangeSet`](/documentation/Swift/RangeSet)

[`FloatingPointSign`](/documentation/Swift/FloatingPointSign)

[`SIMD8`](/documentation/Swift/SIMD8)

[`Int16`](/documentation/Swift/Int16)

[`Int64`](/documentation/Swift/Int64)

[`Ranges-swift.struct`](/documentation/Swift/RangeSet/Ranges-swift.struct)

[`UnitsFormatStyle`](/documentation/Swift/Duration/UnitsFormatStyle)

[`UnsafeMutableRawPointer`](/documentation/Swift/UnsafeMutableRawPointer)

[`UInt128`](/documentation/Swift/UInt128)

[`KeyPath`](/documentation/Swift/KeyPath)

[`UnsafeMutablePointer`](/documentation/Swift/UnsafeMutablePointer)

[`EmptyCollection`](/documentation/Swift/EmptyCollection)

[`Float16`](/documentation/Swift/Float16)

[`Int8`](/documentation/Swift/Int8)

[`UnsafePointer`](/documentation/Swift/UnsafePointer)

[`SIMDMask`](/documentation/Swift/SIMDMask)

[`Character`](/documentation/Swift/Character)

[`Duration`](/documentation/Swift/Duration)

[`Index`](/documentation/Swift/CollectionDifference/Index)

[`AtomicStoreOrdering`](/documentation/Synchronization/AtomicStoreOrdering)

[`Kind-swift.struct`](/documentation/Swift/Unicode/UTF8/ValidationError/Kind-swift.struct)

[`UInt`](/documentation/Swift/UInt)

[`UInt8`](/documentation/Swift/UInt8)

[`AtomicUpdateOrdering`](/documentation/Synchronization/AtomicUpdateOrdering)

[`Task`](/documentation/Swift/Task)

[`Placeholder`](/documentation/Swift/String/LocalizationValue/Placeholder)

[`Float80`](/documentation/Swift/Float80)

[`Change`](/documentation/Swift/CollectionDifference/Change)

[`Continuation`](/documentation/Swift/AsyncStream/Continuation)

[`SIMD3`](/documentation/Swift/SIMD3)

[`Instant`](/documentation/Swift/ContinuousClock/Instant)

[`CollectionOfOne`](/documentation/Swift/CollectionOfOne)

[`Index`](/documentation/Swift/Dictionary/Index)

[`LocalTestingActorID`](/documentation/Distributed/LocalTestingActorID)

[`FloatingPointRoundingRule`](/documentation/Swift/FloatingPointRoundingRule)

[`Dictionary`](/documentation/Swift/Dictionary)

[`ZeroValueUnitsDisplayStrategy`](/documentation/Swift/Duration/UnitsFormatStyle/ZeroValueUnitsDisplayStrategy)

[`Int32`](/documentation/Swift/Int32)

[`UInt16`](/documentation/Swift/UInt16)

[`Keys-swift.struct`](/documentation/Swift/Dictionary/Keys-swift.struct)

[`Index`](/documentation/Swift/ReversedCollection/Index)

[`Attributed-swift.struct`](/documentation/Swift/Duration/TimeFormatStyle/Attributed-swift.struct)

[`SIMD4`](/documentation/Swift/SIMD4)

[`AnyHashable`](/documentation/Swift/AnyHashable)

[`Result`](/documentation/Swift/Result)

[`Index`](/documentation/Swift/Set/Index)

[`Int`](/documentation/Swift/Int)

[`Encoding`](/documentation/Swift/String/Encoding)

[`ClosedRange`](/documentation/Swift/ClosedRange)

[`SIMD2`](/documentation/Swift/SIMD2)

[`ArraySlice`](/documentation/Swift/ArraySlice)

[`ObjectIdentifier`](/documentation/Swift/ObjectIdentifier)

[`OpaquePointer`](/documentation/Swift/OpaquePointer)

[`Bool`](/documentation/Swift/Bool)

[`UnsafeRawPointer`](/documentation/Swift/UnsafeRawPointer)

[`SIMD64`](/documentation/Swift/SIMD64)

[`ByteOrder`](/documentation/Swift/ByteOrder)

[`UTF32`](/documentation/Swift/Unicode/UTF32)

[`Unit`](/documentation/Swift/Duration/UnitsFormatStyle/Unit)

[`Index`](/documentation/Swift/DiscontiguousSlice/Index)

[`String`](/documentation/Swift/String)

[`Termination`](/documentation/Swift/AsyncStream/Continuation/Termination)

[`UnownedTaskExecutor`](/documentation/Swift/UnownedTaskExecutor)

[`FractionalPartDisplayStrategy`](/documentation/Swift/Duration/UnitsFormatStyle/FractionalPartDisplayStrategy)

[`Optional`](/documentation/Swift/Optional)
[`UniqueArray`](/documentation/Swift/UniqueArray)
[`Scalar`](/documentation/Swift/Unicode/Scalar)
[`Float`](/documentation/Swift/Float)
[`DisplayStyle-swift.enum`](/documentation/Swift/Mirror/DisplayStyle-swift.enum)
[`CapitalizationType-swift.enum`](/documentation/Swift/String/IntentInputOptions/
CapitalizationType-swift.enum)
[`RemoteCallTarget`](/documentation/Distributed/RemoteCallTarget)
[`GeneralCategory`](/documentation/Swift/Unicode/GeneralCategory)
[`ValidationError`](/documentation/Swift/Unicode/UTF8/ValidationError)
[`Double`](/documentation/Swift/Double)
[`Comparator`](/documentation/Swift/String/Comparator)
[`AtomicLoadOrdering`](/documentation/Synchronization/AtomicLoadOrdering)

Inherited By

[`SignedInteger`](/documentation/Swift/SignedInteger)
[`BinaryFloatingPoint`](/documentation/Swift/BinaryFloatingPoint)
[`FloatingPoint`](/documentation/Swift/FloatingPoint)
[`FixedWidthInteger`](/documentation/Swift/FixedWidthInteger)
[`InstantProtocol`](/documentation/Swift/InstantProtocol)
[`SIMD`](/documentation/Swift/SIMD)
[`UnsignedInteger`](/documentation/Swift/UnsignedInteger)
[`BinaryInteger`](/documentation/Swift/BinaryInteger)
[`StringProtocol`](/documentation/Swift/StringProtocol)
[`DistributedActor`](/documentation/Distributed/DistributedActor)

Inherits From

[`Equatable`](/documentation/Swift/Equatable)
